

# EnergyGrid

All Interview Questions — Module-wise + Complete Spring Boot

| Emp ID  | Name                          | Backend Module                         | Frontend Module                     |
|---------|-------------------------------|----------------------------------------|-------------------------------------|
| 2507351 | Vinnakota Bala Adithya Vardan | IAM + Energy Analytics                 | Login + Admin + Analytics Dashboard |
| 2507325 | Palleti Nagesh Varma          | Billing & Tariff Management            | Billing Dashboard + Dispute UI      |
| 2507384 | Ponnamanda Raghu Vamsi        | Consumer Account + Meter Data          | Consumer Portal + Meter Reading UI  |
| 2507286 | Vakada Srinu                  | Payments & Collections + Notifications | Payment Portal + Notifications UI   |
| 2507337 | Sai Sowmya Kotturi            | Outage & Grid Event Management         | Grid Ops Dashboard + Outage Tracker |

## MODULE 1 — Vinnakota Bala Adithya Vardan | IAM + Energy Analytics & Reporting

### BACKEND QUESTIONS — IAM + Energy Analytics

- Q1.** Walk me through your IAM module's complete database schema — all three tables with columns and data types.
- Q2.** Why did you use 6 roles (CONSUMER, FIELD\_TECH, BILLING, GRID\_OPS, COMPLIANCE, ADMIN) instead of a simpler role structure?
- Q3.** Explain how JWT authentication works end-to-end in your project — from login request to secured API access.
- Q4.** What is BCrypt? Why is it used for password hashing and what makes it more secure than MD5 or SHA?
- Q5.** What does @Enumerated(EnumType.STRING) do and why should you never use EnumType.ORDINAL?
- Q6.** What is AuditLogService and why is it designed as a shared method called by all other modules?
- Q7.** How does EnergyReportService generate its reports? What does the Metrics TEXT column contain?
- Q8.** What JUnit tests did you write for UserService? Name each test and what it verifies.
- Q9.** What does SecurityConfig do? Explain each setting — CSRF, session management, CORS, filter order.
- Q10.** Explain the custom query methods in UserRepository and AuditLogRepository. How does Spring Data generate SQL from method names?
- Q11.** What is OncePerRequestFilter and why does JwtFilter extend it instead of implementing Filter directly?
- Q12.** How does JwtUtil.generateToken() build the JWT? What are the three parts of a JWT?
- Q13.** What is BCryptPasswordEncoder.matches() and how does it compare passwords without storing plain text?
- Q14.** What is @CreationTimestamp in AuditLog.timestamp? How is it different from setting LocalDateTime.now() in service code?
- Q15.** How does the AuditLogRepository support paginated queries? What does Page findAll(Pageable) return?
- Q16.** What would happen if AuditLogServiceImpl was NOT excluded from the AOP LoggingAspect pointcut?
- Q17.** What is the difference between findByEmail() returning Optional vs List?
- Q18.** How does EnergyReportController's GET /api/energy-reports/summary endpoint aggregate KPIs?

### FRONTEND QUESTIONS — Login + Admin Console + Analytics Dashboard

- Q1.** Explain how AuthContext.jsx works — createContext, Provider, and the useAuth custom hook.
- Q2.** What is ProtectedRoute.jsx and how does it redirect unauthenticated users?
- Q3.** How did you implement role-based sidebar navigation in Layout.jsx?
- Q4.** How are the 4 KPI cards in AnalyticsDashboard populated — Phase 1 mock data vs Phase 3 real API?
- Q5.** Explain how the axios interceptor in api.js automatically appends the JWT token to every request.
- Q6.** How does login flow work: user submits form → JWT received → stored → redirect by role?
- Q7.** What is localStorage and why do you store the JWT token there instead of in React state?
- Q8.** How does UserListPage implement search and filter by role using useState?
- Q9.** Explain UserFormModal — how is it a controlled component and how does it add a user to state on submit?
- Q10.** What is useNavigate() and how do you use it to redirect after login based on the user's role?
- Q11.** How do loading spinners work in AnalyticsDashboard while the API call is pending?
- Q12.** What is a response interceptor in axios and how would you use it to handle 401 (token expired)?

## MODULE 2 — Palleti Nagesh Varma | Billing & Tariff Management

### BACKEND QUESTIONS — Billing & Tariff Management

- Q1.** Explain the Tariff entity structure in detail — especially the SlabRates TEXT JSON field and why you chose TEXT over a separate table.
- Q2.** Walk me through generateBill() step by step — how is TotalAmount calculated from units consumed and tariff slabs?
- Q3.** What does the batch endpoint PATCH /api/bills/mark-overdue do? How is it implemented in BillingService?
- Q4.** Describe the complete BillingDispute lifecycle — all 4 states with transitions and business rules.
- Q5.** Explain @ManyToOne and @JoinColumn in the Bill entity. What SQL does Hibernate generate?
- Q6.** Why does BillingDispute store consumerId as Integer (no FK) instead of @ManyToOne to User entity?
- Q7.** What are the custom query methods in BillRepository? Explain findByTariff\_TariffId() and the underscore notation.
- Q8.** What JUnit tests did you write for BillingService? Name each test, the mock setup, and what it asserts.
- Q9.** What JUnit tests did you write for BillingDisputeService? Explain the alreadyResolved exception test.
- Q10.** What is @Enumerated(EnumType.STRING) on BillStatus? What are all 5 bill status values?
- Q11.** How does TariffService.getActiveTariffByCategory() use the repository to find the correct tariff for bill generation?
- Q12.** What happens if a consumer raises a dispute on a bill that is already RESOLVED? How does your service handle it?
- Q13.** How does BillingService.markOverdue() use LocalDate.now() to find expired bills?
- Q14.** What is the difference between BillingService and BillingDisputeService responsibilities?
- Q15.** How does the PATCH /api/tariffs/{id}/status endpoint work? What validations does it perform?
- Q16.** What is TariffCategory enum? How is it used to filter tariffs in GET /api/tariffs?accategory=?
- Q17.** Why do you use @Slf4j in all service classes? Show an example log statement from BillingService.

### FRONTEND QUESTIONS — Billing Dashboard + Dispute Management UI

- Q1.** How does GenerateBillModal auto-calculate the estimated amount as the user fills in Units and selects a Tariff?
- Q2.** Explain the 4 summary cards in BillListPage — how are Total, Paid, Overdue, and Disputed counts computed from state?
- Q3.** How do Bootstrap status badges work for bill status? Explain the getStatusVariant() helper function.
- Q4.** How does the Resolve Dispute button use window.confirm() and then update local state on success?
- Q5.** How did you implement the Mark Overdue button to batch-update bills past their due date in state?
- Q6.** Explain TariffListPage — what columns does the table show and how is it populated from the API?
- Q7.** How does the Tariff select dropdown in GenerateBillModal get populated? What API call does it make?
- Q8.** What is the difference between Phase 1 mock data and Phase 3 API integration in BillListPage?
- Q9.** How do you handle API errors (e.g., 400 Bad Request) in the Generate Bill form and show them to the user?
- Q10.** Explain BillingDisputePage — how does it filter to show only OPEN disputes by default?
- Q11.** How does the status filter dropdown on BillListPage work — filter by GENERATED, PAID, OVERDUE?

## MODULE 3 — Ponnamanda Raghu Vamsi | Consumer Account + Connection + Meter Data

### BACKEND QUESTIONS — Consumer Account + Connection + Meter Data

- Q1.** Explain all 4 database tables in your module — ConsumerAccount, ConnectionRequest, Meter, MeterReading — with all columns.
- Q2.** How does the ConsumerAccount entity relate to the Meter entity in JPA? Why does ConsumerAccount store meterId as Integer instead of @OneToMany?
- Q3.** Explain MeterReadingService.calculateConsumption() — what query does it use and how is kWh calculated?
- Q4.** What is findTop1ByMeter\_MeterIdOrderByReadingDateDesc() — break down each part of this method name and the SQL it generates.
- Q5.** Explain the ConnectionRequest workflow step by step — from consumer submission to technician assignment to completion.
- Q6.** What are all 8 enums in your module? Why is it important to have so many distinct enums?
- Q7.** What does MeterService.installMeter() validate before creating the Meter entity? What exceptions does it throw?
- Q8.** What JUnit tests did you write for ConsumerAccountService and MeterReadingService? Name each and what it verifies.
- Q9.** What is the difference between MeterType ANALOG, DIGITAL, and SMARTAMI? How does it affect readings?
- Q10.** What is ReadingSource MANUAL vs AMI vs ESTIMATED? Which source does BillingService trust for bill generation?
- Q11.** How does ConnectionRequestService.assignTechnician() validate that the assigned user actually has FIELD\_TECH role?
- Q12.** What is findByMeter\_MeterIdAndReadingDateBetween() and when is it called?
- Q13.** How does MeterService handle meter replacement — old meter REPLACED, new meter ACTIVE, ConsumerAccount updated?
- Q14.** What is @ManyToOne @JoinColumn(name='AccountID') in Meter entity? What SQL does it create?
- Q15.** What is the PATCH /api/connection-requests/{id}/assign?techId= endpoint's full request/response flow?
- Q16.** How does ConsumerAccountRepository.findByZoneld(Integer zoneld) help the outage module?
- Q17.** Why is AccountID stored as FK in Meter but AssignedTechID stored as Integer (no FK) in ConnectionRequest?

### FRONTEND QUESTIONS — Consumer Portal + Meter Reading UI

- Q1.** How does consumption calculation work in MeterReadingPage — what JavaScript computes (latestReading - previousReading)?
- Q2.** How did you implement the Assign Technician button — dropdown population, state update on success?
- Q3.** How does ConsumerListPage display connection type and status badges using Bootstrap?
- Q4.** How does ConnectionRequestPage filter to show only PENDING requests by default?
- Q5.** Explain MeterReadingFormModal as a controlled component — all form fields and their state variables.
- Q6.** How does the meter selector dropdown in MeterReadingPage trigger a data fetch on change?
- Q7.** How does ConsumerFormModal validate that Zoneld is a number before submitting?
- Q8.** What happens in state when a ConnectionRequest is assigned a technician — how does the row update?
- Q9.** How do you show 'N/A' for consumption when fewer than 2 readings exist for a meter?
- Q10.** How does the 'Add Consumer Account' flow work end-to-end from button click to table row appearing?

## MODULE 4 — Vakada Srinu | Payments & Collections + Notifications

### BACKEND QUESTIONS — Payments & Collections + Notifications

- Q1.** Why is `recordPayment()` annotated with `@Transactional`? What happens if you remove it and the second DB operation fails?
- Q2.** What validations does `POST /api/payments` perform before recording a payment? What HTTP status codes are returned?
- Q3.** Explain the `InstallmentPlan` use case — when is it created, what are `ACTIVE/COMPLETED/DEFAULTED` states?
- Q4.** Describe all `NotificationService` methods — `sendNotification`, `getByUser`, `markAsRead`, `markAllAsRead`, `getUnreadCount`.
- Q5.** What is `countByUserIdAndStatus()` in `NotificationRepository`? What SQL does Spring Data generate for it?
- Q6.** Explain `PATCH /api/notifications/user/{userId}/read-all` — service logic and the `{updated: N}` response format.
- Q7.** What are the 5 notification categories (`BILLING`, `PAYMENT`, `OUTAGE`, `METER`, `COMPLIANCE`)? When is each triggered?
- Q8.** What JUnit tests did you write for `PaymentService`? Name all 3 tests and what each mocks and asserts.
- Q9.** What JUnit tests did you write for `NotificationService`? Name all 4 tests.
- Q10.** What is `@Transactional propagation=REQUIRES_NEW`? How could it be applied to `NotificationService` for reliability?
- Q11.** Why do `Payment` and `Bill` have no direct `@ManyToOne` relationship — why store `billId` as `Integer`?
- Q12.** What is `PaymentMethod` enum? What are the 4 payment methods and when is each used?
- Q13.** How does `recordPayment()` update the `Bill` status to `PAID` atomically with saving the `Payment`?
- Q14.** What is `InstallmentPlanRepository.findByAccountId()` used for? When is a plan marked `DEFAULTED`?
- Q15.** What happens if `GET /api/notifications/user/{userId}/count` is called for a user with zero notifications?
- Q16.** How does `NotificationService.getByUser()` order notifications — what SQL `ORDER BY` is used?
- Q17.** What is the difference between `NotificationStatus` `READ`, `UNREAD`, and `DISMISSED`?

### FRONTEND QUESTIONS — Payment Portal + Notifications UI

- Q1.** How does `NotificationBell` polling work using `setInterval`? Why must you call `clearInterval` in the `useEffect` cleanup?
- Q2.** How do the 6 category filter tabs in `NotificationsPage` work — how are they filtered client-side without extra API calls?
- Q3.** How does the `Mark All Read` button update local state immediately without re-fetching from the API?
- Q4.** Explain `RecordPaymentModal` — form fields, controlled state variables, and what happens on submit.
- Q5.** How does `PaymentListPage` show `Method` and `Status` badges using Bootstrap color classes?
- Q6.** How does the `Installment Plan` form validate that `EndDate` is after `StartDate` before submit?
- Q7.** What is optimistic UI update and how did you use it after marking all notifications read?
- Q8.** How does the unread count badge on `NotificationBell` disappear when all notifications are read?
- Q9.** How does `NotificationsPage` handle the loading state while fetching notifications?
- Q10.** How does the `PaymentListPage` filter payments by account — what query param does it send?
- Q11.** How do you show a success toast after a payment is successfully recorded?

## MODULE 5 — Sai Sowmya Kotturi | Outage & Grid Event Management

### BACKEND QUESTIONS — Outage & Grid Event Management

- Q1.** Explain all fields in the OutageEvent entity — ZoneID, AffectedAccounts, Cause enum, StartTime, EstimatedRestoration, ActualRestoration, Status.
- Q2.** How does the SLA check endpoint GET /api/outage-events/{id}/sla work? What is the slaMet boolean calculation?
- Q3.** Why is FieldDispatchService annotated @Transactional? What does dispatchTechnician() validate before saving?
- Q4.** Explain restoreOutage() — the business rule check, what fields are set, and what HTTP status is returned on error.
- Q5.** What is findByStartTimeBetween(LocalDateTime from, LocalDateTime to) and which other module uses it?
- Q6.** Explain FieldDispatch status progression: DISPATCHED → ONSITE → RESOLVED. What does resolveDispatch() set?
- Q7.** What are all 7 JUnit tests you wrote across OutageEventService and FieldDispatchService? Name each test.
- Q8.** What is OutageCause enum? What are the 4 values and what real-world scenario does each represent?
- Q9.** How does PATCH /api/outage-events/{id}/restore return HTTP 400 when the outage is already RESTORED?
- Q10.** What is @ManyToOne @JoinColumn(name='OutageID') in FieldDispatch entity? What FK does it create?
- Q11.** How does GET /api/outage-events/active return only ACTIVE outages — what repository method is used?
- Q12.** What is @CreationTimestamp on FieldDispatch.dispatchTime? Why is it better than setting LocalDateTime.now() manually?
- Q13.** How does FieldDispatchService.resolveDispatch() set both status=RESOLVED and resolutionTime atomically?
- Q14.** What is findByZoneId(Integer zoneId) used for in OutageEventRepository?
- Q15.** How does the outage module integrate with NotificationService to alert consumers in an affected zone?
- Q16.** What happens to FieldDispatch records when an outage is already RESTORED — can you still dispatch a technician?
- Q17.** How does the SLA endpoint handle the case where ActualRestoration is null (outage still ACTIVE)?

### FRONTEND QUESTIONS — Grid Ops Dashboard + Outage Tracker

- Q1.** How do contextual status buttons work in FieldDispatchPage — different buttons rendered for DISPATCHED vs ONSITE vs RESOLVED?
- Q2.** How is the summary row (Active Outages, Total Affected Accounts, Avg Resolution Time) calculated using JavaScript array methods?
- Q3.** How does the Restore button use window.confirm() and then update state on success?
- Q4.** How are ACTIVE and RESTORED outages visually differentiated using Bootstrap badge colors?
- Q5.** Explain ReportOutageModal — all form fields, EstimatedRestoration datetime input, and state management.
- Q6.** How does DispatchFormModal work — what fields does it have and what API call does it make?
- Q7.** How does OutageListPage filter by status (ACTIVE/RESTORED/INVESTIGATING) using a dropdown?
- Q8.** How does the Avg Resolution Time calculation handle outages that are still ACTIVE (null actualRestoration)?
- Q9.** How does the status update button in FieldDispatchPage use prev.map() to update only the changed row?
- Q10.** How do you prevent a user from clicking Restore twice — what check do you add in the click handler?

## GENERAL BACKEND QUESTIONS — Spring Boot, JPA, REST, Security, AOP

### Spring Boot Core

- Q1. What is Spring Boot and how is it different from Spring Framework?
- Q2. What is `@SpringBootApplication` and what three annotations does it combine?
- Q3. What is `application.properties`? What did you configure in `EnergyGrid`?
- Q4. What is `@Value('${key}')` and how did you use it for JWT configuration?
- Q5. What is the difference between `@Entity` and a DTO? Why keep them separate?
- Q6. What are Spring Boot Starters? Name the starters used in `EnergyGrid`.
- Q7. What is Spring Boot auto-configuration? How do `@ConditionalOnClass` annotations work?
- Q8. What is the embedded Tomcat? How do you change the port?
- Q9. What is `spring.jpa.hibernate.ddl-auto`? Explain all 5 options (create, update, validate, none, create-drop).
- Q10. What is `CommandLineRunner` and when would you use it for data seeding?
- Q11. What is `@Bean` and when do you use it instead of `@Component`?
- Q12. What is Spring Profiles? How would you configure dev, test, and prod profiles in `EnergyGrid`?
- Q13. What is Spring Boot Actuator? Name 5 endpoints and what they expose.
- Q14. What is the complete lifecycle of an HTTP request in Spring Boot — from Tomcat to controller to response?
- Q15. What is Lombok? Which annotations did you use and what code do they generate?
- Q16. What is `@Slf4j`? How does logging work — log levels, parameterised messages, configuration?

### JPA and Hibernate

- Q1. What is Spring Data JPA? What methods does `JpaRepository` give you out of the box?
- Q2. What is EAGER vs LAZY loading? What is the N+1 problem and how do you fix it?
- Q3. Explain derived query methods — how does Spring Data generate SQL from method names?
- Q4. What is `@Transactional`? Explain propagation (REQUIRED, REQUIRES\_NEW), `readOnly`, and `rollbackFor`.
- Q5. What is `@Transactional` on a private method or self-invocation — does it work? Why not?
- Q6. What is `@CreationTimestamp` vs `@UpdateTimestamp`? When would you use each?
- Q7. What is the difference between `save()` and `saveAndFlush()` in `JpaRepository`?
- Q8. What is `@Enumerated(EnumType.STRING)` vs `ORDINAL`? Why is `ORDINAL` dangerous?
- Q9. What is JPQL and how does it differ from SQL? Give an example query.
- Q10. How does pagination work in Spring Data JPA? Explain `Pageable`, `PageRequest`, and `Page`.
- Q11. What is `@ManyToOne` and `@JoinColumn`? What FK column does it create in the database?
- Q12. What is the difference between `findById()` returning `Optional` and `getReferenceById()` returning a proxy?
- Q13. What is `@Modifying` and when is it needed with `@Query`?
- Q14. What is `ddl-auto=update` vs `validate` — when would you use each in production?
- Q15. What is Hibernate's dirty checking and when does a flush happen?
- Q16. What is `@Query(nativeQuery=true)`? When would you use native SQL over JPQL?

### REST API and Spring MVC

- Q1. What is `@RestController`? How is it different from `@Controller`?



- Q2. What is the difference between `@PathVariable`, `@RequestParam`, and `@RequestBody`?
- Q3. What is the difference between `@PutMapping` and `@PatchMapping`? When should you use each?
- Q4. What HTTP status codes did you use in EnergyGrid and what is the semantic of each?
- Q5. How does `GlobalExceptionHandler` work using `@RestControllerAdvice` and `@ExceptionHandler`?
- Q6. What is `ResponseEntity`? Why use it instead of returning an object directly from the controller?
- Q7. What is `@RequestMapping` at class level vs method level?
- Q8. How do you return HTTP 201 Created with the location header after a POST creates a resource?
- Q9. What is content negotiation in Spring MVC? How does Jackson serialise your DTOs to JSON?
- Q10. What is `@Valid` and Bean Validation? How do you return field-level validation errors to the client?
- Q11. What is the difference between `@ControllerAdvice` and `@RestControllerAdvice`?
- Q12. How do you test REST controllers using `MockMvc` in `@WebMvcTest`?

### Spring Security and JWT

- Q1. How does Spring Security's filter chain work? What is `DelegatingFilterProxy`?
- Q2. What is a JWT? What are its three parts and how is the signature validated?
- Q3. What is CSRF and why did you disable it in EnergyGrid?
- Q4. What is `OncePerRequestFilter` and why does `JwtFilter` extend it?
- Q5. What is `SecurityContextHolder` and how does `JwtFilter` populate it?
- Q6. What is CORS and how did you configure it to allow the React frontend to call the API?
- Q7. What is stateless session management (`SessionCreationPolicy.STATELESS`)?
- Q8. What is method-level security (`@PreAuthorize`, `@Secured`)? How would you add it to EnergyGrid?
- Q9. What is BCrypt and why does it use a salt? What is the cost factor?
- Q10. How does `UsernamePasswordAuthenticationToken` work in Spring Security?
- Q11. What is the difference between authentication and authorisation?
- Q12. How does `permitAll()` vs `authenticated()` work in the `SecurityFilterChain`?

### Spring AOP, Testing, and Advanced

- Q1. What is AOP? Explain Aspect, Advice, Pointcut, JoinPoint, and Weaving.
- Q2. What is the difference between `@Before`, `@After`, `@AfterReturning`, `@AfterThrowing`, and `@Around`?
- Q3. What is the pointcut expression `execution(* com.cts.energygrid.service.impl.*(..))` — explain each part?
- Q4. Why did you exclude `AuditLogServiceImpl` from the `LoggingAspect` pointcut?
- Q5. What is circular dependency in Spring? How do you fix it?
- Q6. How does JUnit 5 + Mockito testing work? Explain `@Mock`, `@InjectMocks`, `when()`, `verify()`.
- Q7. What is the difference between `@Mock` and `@MockBean`?
- Q8. What is `@SpringBootTest` vs `@WebMvcTest` vs `@DataJpaTest`? When do you use each?
- Q9. What is `@ExtendWith(MockitoExtension.class)` — what does it enable?
- Q10. What is `ArgumentCaptor` in Mockito and when do you use it?
- Q11. What is `assertThrows()` in JUnit 5 and how did you use it for exception tests?
- Q12. What is `@BeforeEach` and when do you use it in a test class?



## GENERAL FRONTEND QUESTIONS — React, Axios, Bootstrap, JavaScript

### React Core and Hooks

- Q1. What is React? Explain its core concepts: component, JSX, props, state, virtual DOM, unidirectional data flow.
- Q2. What is the difference between `useState` and `useEffect`? Give examples from EnergyGrid.
- Q3. What is the dependency array in `useEffect` — empty `[]`, specific values, and no array? What does each do?
- Q4. What is the cleanup function in `useEffect`? How did you use it in NotificationBell?
- Q5. What are controlled vs uncontrolled components? Give an example of each.
- Q6. What is React Context API? Explain `createContext`, `Provider`, and `useContext`.
- Q7. What is prop drilling and how does Context API solve it? Give an example from EnergyGrid.
- Q8. What is a custom hook? Explain `useAuth()` — why is it a custom hook and what does it return?
- Q9. What are the Rules of Hooks? Why can't you call hooks inside a loop or if-statement?
- Q10. What is the Virtual DOM and how does React's diffing algorithm minimise real DOM updates?
- Q11. What is `useState` initialiser function — `useState(() => heavyComputation())` — when would you use it?
- Q12. What is `useCallback` and `useMemo`? When would you use them for performance optimisation?
- Q13. What is `React.memo()` and when would you use it in EnergyGrid?
- Q14. What is key prop in a list — why is it important and what happens without it?
- Q15. What is lifting state up? Give an example from your module pages.
- Q16. What is conditional rendering in React? Show 3 different ways to render conditionally.

### React Router, Axios, and API Integration

- Q1. How does React Router DOM v6 work? Explain `BrowserRouter`, `Routes`, `Route`, and nested routes.
- Q2. What is `ProtectedRoute` and how does it redirect unauthenticated users?
- Q3. What is `useNavigate()` and how does it differ from the `Link` component?
- Q4. How did you configure the axios instance with `baseUrl` and the request interceptor?
- Q5. What is the response interceptor in axios — how do you handle global 401 (token expired)?
- Q6. How do you make an API call on component mount — `useEffect` with empty dependency array?
- Q7. How do you implement loading and error states for API calls in React?
- Q8. What is `async/await` in JavaScript? How did you use it in service functions?
- Q9. What is `Promise.catch()` vs `try/catch` in async functions? Which pattern did you use?
- Q10. How do you pass query parameters in axios — `axios.get('/api/bills', {params: {accountId: 5}})`?
- Q11. What is CORS and why do you get CORS errors when calling the Spring Boot API from React?
- Q12. How do you send a PATCH request with axios? Show syntax for `PATCH /api/bills/{id}/status`.

### Bootstrap 5 and JavaScript

- Q1. What is Bootstrap 5? How do you integrate it into a React project?
- Q2. How does the Bootstrap grid system work — `Container`, `Row`, `Col`, and breakpoints?
- Q3. What Bootstrap components did you use in EnergyGrid? Describe at least 6.
- Q4. How do Bootstrap badges work? How did you use them for bill and outage status color coding?
- Q5. How does a Bootstrap Modal work in React — show/hide controlled by React state?
- Q6. What is JavaScript `Array.filter()` — how did you use it for bill status summary cards?

- Q7.** What is `Array.reduce()` — how did you use it to sum `AffectedAccounts` across outages?
- Q8.** What is `Array.map()` — how did you use it to update one item in a state array immutably?
- Q9.** What is the spread operator `{...obj}` and why is it important for immutable state updates?
- Q10.** What is optional chaining `(?.)` — how does `error.response?.data?.message` prevent crashes?
- Q11.** What is `localStorage` — what methods does it have and what are its limitations?
- Q12.** What is `JSON.parse()` and `JSON.stringify()` — when do you need them with `localStorage`?
- Q13.** What is `window.confirm()` — when did you use it in `EnergyGrid` and why?

## ALL SPRING BOOT QUESTIONS — Complete List for Interview Preparation

### Spring Boot Fundamentals (Q1–Q20)

- Q1. What is Spring Boot? What problems does it solve over plain Spring Framework?
- Q2. What is `@SpringBootApplication`? What are the 3 annotations it combines?
- Q3. What is Spring IoC (Inversion of Control) Container? How does it work?
- Q4. What is Dependency Injection? What are the 3 types and which is recommended?
- Q5. What is the difference between `@Component`, `@Service`, `@Repository`, and `@Controller`?
- Q6. What is `@Bean`? How is it different from `@Component`? When do you use it?
- Q7. What is `@Autowired`? When do you also need `@Qualifier` or `@Primary`?
- Q8. What are Bean Scopes — Singleton, Prototype, Request, Session? Which is the default?
- Q9. What is `application.properties` vs `application.yml`? What is the difference?
- Q10. What is `@Value('${key}')`? How do you use it? What is `@ConfigurationProperties`?
- Q11. What is Spring Boot auto-configuration? How does `@ConditionalOnClass` work?
- Q12. What are Spring Boot Starters? What does `spring-boot-starter-web` include?
- Q13. What is embedded server in Spring Boot? How do you switch from Tomcat to Jetty?
- Q14. What is `spring.jpa.hibernate.ddl-auto`? Explain create, update, validate, none, create-drop.
- Q15. What is Spring Profiles? How do you define and activate dev/prod/test profiles?
- Q16. What is Spring Boot Actuator? Name 5 endpoints and what monitoring data they expose.
- Q17. What is `CommandLineRunner` vs `ApplicationRunner`? When would you use them?
- Q18. What is the full Spring MVC request lifecycle from HTTP request to JSON response?
- Q19. What is `ResponseEntity`? When do you return it vs returning a plain object?
- Q20. What is `@SpringBootTest` vs `@WebMvcTest` vs `@DataJpaTest`? When do you use each?

### Spring Data JPA and Hibernate (Q21–Q40)

- Q1. What is JPA and Hibernate? What is the role of each?
- Q2. What is Spring Data JPA? What does extending `JpaRepository` give you?
- Q3. What is `@Entity`, `@Table`, `@Id`, `@GeneratedValue`? Explain each annotation.
- Q4. What is `@Column` — nullable, unique, length, name? Give examples.
- Q5. What is `@ManyToOne` vs `@OneToMany` vs `@ManyToMany` vs `@OneToOne`?
- Q6. What is `@JoinColumn`? What FK column does it create in the database?
- Q7. What is EAGER vs LAZY loading? What are the defaults for `@ManyToOne` and `@OneToMany`?
- Q8. What is the N+1 query problem? Give an example and explain 3 ways to solve it.
- Q9. What is `@Transactional`? Explain propagation, isolation, readOnly, and rollbackFor.
- Q10. What is `@Transactional` on a private method or self-invocation — does it work and why?
- Q11. What is derived query methods in Spring Data? Explain the naming convention with 5 examples.
- Q12. What is `@Query(JPQL)`? How does it differ from `@Query(nativeQuery=true)`?
- Q13. What is `@Modifying`? When is it required alongside `@Query`?
- Q14. What is Pageable, PageRequest, and Page? How do you implement pagination?
- Q15. What is `@Enumerated(EnumType.STRING)` vs `ORDINAL`? Why is `ORDINAL` dangerous?
- Q16. What is `@CreationTimestamp` and `@UpdateTimestamp` in Hibernate?

- Q17. What is the difference between `save()` and `saveAndFlush()` in `JpaRepository`?
- Q18. What is `findById()` returning `Optional` vs `getReferenceById()` returning a proxy?
- Q19. What is JPQL? How does it differ from SQL? Give a JPQL example.
- Q20. What is Hibernate dirty checking? When does a flush happen automatically?

### Spring Security (Q41–Q55)

- Q1. What is Spring Security? How does the Security Filter Chain work?
- Q2. What is `DelegatingFilterProxy` and how does it connect servlet filters with Spring beans?
- Q3. What is JWT — JSON Web Token? Explain the 3 parts: header, payload, signature.
- Q4. How do you validate a JWT — signature check, expiry check, claim extraction?
- Q5. What is `OncePerRequestFilter`? Why should security filters extend it?
- Q6. What is `SecurityContextHolder`? How does `JwtFilter` populate it?
- Q7. What is `UsernamePasswordAuthenticationToken`? What are its constructor arguments?
- Q8. What is CSRF? Why is it disabled in stateless REST APIs using JWT?
- Q9. What is CORS? How do you configure it in Spring Boot to allow cross-origin requests?
- Q10. What is `SessionCreationPolicy.STATELESS`? What does it prevent?
- Q11. What is `BCryptPasswordEncoder`? What is the cost factor and why does it matter?
- Q12. What is method-level security? Explain `@PreAuthorize`, `@PostAuthorize`, `@Secured`.
- Q13. What is the difference between authentication and authorisation in Spring Security?
- Q14. What is `permitAll()` vs `authenticated()` vs `hasRole()` in `SecurityFilterChain`?
- Q15. What is `@EnableWebSecurity` and `@EnableMethodSecurity`? What do they enable?

### Spring AOP (Q56–Q65)

- Q1. What is AOP (Aspect-Oriented Programming)? What problem does it solve?
- Q2. Explain the AOP terms: Aspect, Advice, JoinPoint, Pointcut, and Weaving.
- Q3. What is the difference between `@Before`, `@After`, `@AfterReturning`, `@AfterThrowing`, `@Around`?
- Q4. What is a pointcut expression — `execution(* com.example.service.*(..))`? Break it down.
- Q5. What is `@Around` advice? How do you call the original method from it using `proceed()`?
- Q6. How does Spring AOP use CGLIB proxies? What limitation does this have for self-invocation?
- Q7. What is the difference between Spring AOP and AspectJ?
- Q8. How do you exclude specific beans from an AOP pointcut expression?
- Q9. What is `proceedingJoinPoint.getSignature().getName()`? When is it useful in logging?
- Q10. What are real-world use cases for AOP beyond logging — transaction management, security, caching?

### JUnit 5 + Mockito (Q66–Q80)

- Q1. What is the difference between unit testing and integration testing? Which did you write?
- Q2. What is `@ExtendWith(MockitoExtension.class)`? What does it enable?
- Q3. What is `@Mock`? What default values does a Mockito mock return?
- Q4. What is `@InjectMocks`? How does Mockito inject `@Mock` fields into the class under test?
- Q5. What is `@Spy`? How is it different from `@Mock`?
- Q6. How does `when(mock.method(arg)).thenReturn(value)` work?
- Q7. How do you stub void methods in Mockito — `doNothing()`, `doThrow()`?

- Q8.** What is `verify(mock, times(1)).method(arg)`? What does it assert?
- Q9.** What is `ArgumentCaptor`? Give an example use case.
- Q10.** What is `assertThrows(Exception.class, () -> ...)` in JUnit 5?
- Q11.** What is `@BeforeEach` and `@AfterEach`? Give an example setup scenario.
- Q12.** What is the difference between `@Mock` and `@MockBean`?
- Q13.** What is `@SpringBootTest(webEnvironment=RANDOM_PORT)`? When do you use `TestRestTemplate`?
- Q14.** What is `@WebMvcTest`? How do you use `MockMvc` to test a controller?
- Q15.** What is `@DataJpaTest`? What database does it use and what beans does it load?

### Microservices and Docker (Q81–Q95)

- Q1.** What is the difference between monolithic architecture and microservices?
- Q2.** What are the advantages of microservices? What are the challenges?
- Q3.** What is an API Gateway? What does it handle — routing, auth, rate limiting?
- Q4.** What is Service Discovery? How does Netflix Eureka work?
- Q5.** What is Spring Cloud? Name 3 Spring Cloud components and their purposes.
- Q6.** What is load balancing? What is the difference between client-side and server-side load balancing?
- Q7.** What is a distributed transaction in microservices? How do you handle it without `@Transactional`?
- Q8.** What is Docker? What is the difference between a Docker image and a container?
- Q9.** What is a Dockerfile? Explain each instruction: `FROM`, `WORKDIR`, `COPY`, `EXPOSE`, `ENTRYPOINT`.
- Q10.** What is Docker Compose? How would you orchestrate EnergyGrid app + MySQL with it?
- Q11.** What is the difference between `EXPOSE` in Dockerfile and `-p` in `docker run`?
- Q12.** What is a Docker volume? Why is it needed for a MySQL container?
- Q13.** What is the 12-factor app methodology? Name at least 5 factors.
- Q14.** What is Spring Boot DevTools? What features does it provide for development?
- Q15.** What is SonarQube? What code quality metrics does it measure?

## QUESTION COUNT SUMMARY

### Total Questions by Section

| Section      | Topic                                                            | BE Qs      | FE Qs     | Total      |
|--------------|------------------------------------------------------------------|------------|-----------|------------|
| Module 1     | Vinnakota — IAM + Energy Analytics                               | 18         | 12        | 30         |
| Module 2     | Palleti — Billing & Tariff Management                            | 17         | 11        | 28         |
| Module 3     | Raghu Vamsi — Consumer Account + Meter                           | 17         | 10        | 27         |
| Module 4     | Vakada Srinu — Payments + Notifications                          | 17         | 11        | 28         |
| Module 5     | Sai Sowmya — Outage & Grid Event                                 | 17         | 10        | 27         |
| Section 3    | General Backend (Spring Boot, JPA, REST, Security, AOP, Testing) | 68         | —         | 68         |
| Section 4    | General Frontend (React, Axios, Bootstrap, JS)                   | —          | 41        | 41         |
| Section 5    | All Spring Boot Questions (Fundamentals to Docker)               | 95         | —         | 95         |
| <b>TOTAL</b> |                                                                  | <b>249</b> | <b>95</b> | <b>344</b> |

**HOW TO USE THIS GUIDE:** Study each person's module questions first so you can confidently explain your own module. Then study the other modules so you understand the full system. Practice general backend and frontend questions with answers. Finally, go through all Spring Boot questions — these are asked in almost every interview regardless of the project. The best answers always include a reference back to EnergyGrid: say 'In my project I used X because Y'.